

Stabilizing an actuated inverted pendulum

In this part of the project, you will design a controller that operates in discrete-time to stabilize an actuated pendulum about its unstable equilibrium corresponding to upright (inverted) configuration. The physical pendulum evolves continuously in time. However, the controller receives measurements of the state and applies a control input at regular intervals.

The physical pendulum is implemented using Simscape Multibody and is provided as part of the Simulink model. Your task is to construct the appropriate models needed for control and observer design. You will design a LQR that stabilizes the system at the unstable equilibrium and design an observer when the full state is not measurable. You will need to implement these in Simulink, reason about selecting appropriate parameters, and evaluate the performance of the control system.

At the end of this part, you should be able to show a working simulation of a control system stabilizing the pendulum using only the angle measurements and under disturbances. You should also be able to justify your reasoning and your design choices.

1 Pendulum model

Consider a torque actuated pendulum with the system dynamics given in continuous-time by

$$J\ddot{\theta} = mgl \sin \theta - b\dot{\theta} + u, \quad (1)$$

in which

- θ is the angle measured from the inverted configuration of the pendulum with positive angles measured counterclockwise from 0, and $\dot{\theta}, \ddot{\theta}$ correspond to the first and the second derivative of θ with respect to time,
- m is the pendulum mass,
- l is the distance from the axis of rotation to the center of mass,
- $J = ml^2$ is the moment of inertia about the axis of rotation,
- b is the viscous damping coefficient,
- g is the gravitational acceleration, and
- u is the actuator torque corresponding to the control input.

1.1 Physical model

The first part of the project requires you to write a MATLAB script to simulate the continuous-time dynamics of the pendulum. This corresponds to solving initial value problems for the given ordinary differential equation.

You can use the MATLAB function

```
[t,y] = ode45(odefun,tspan,y0)
```

to solve the differential equations describing the pendulum dynamics given by Equation (1). See MATLAB documentation for `ode45` for implementation details. Note that MATLAB ODE solvers expect a system of first-order differential equations so you need to put the dynamics in state-space form

$$\dot{x} = f(x, u)$$

with $x = (x_1, x_2)$ and $x_1 = \theta$ and $x_2 = \dot{\theta}$ as the state variables.

Experiment: Assume $u = 0$ and consider different initial conditions. Specifically, simulate the system corresponding to the initial condition $\theta(0) = 0$ and $\dot{\theta}(0) = 0$ and consider small perturbations to the initial state.

1.2 Linear approximation

Derive a linear approximation of the nonlinear dynamics near $(\theta, \dot{\theta}) = 0$ and $u = 0$. For small angles you can assume that $\sin \theta \approx \theta$.

Determine the linear system dynamics as

$$\dot{x} = A_c x + B_c u,$$

in which $x = (x_1, x_2) = (\theta, \dot{\theta})$ is the state and A_c and B_c are the matrices corresponding to the linearized system.

Analysis: Analyze the stability of the linear approximation. How does it relate to the behavior of the pendulum near the equilibrium point corresponding to the upright configuration of the pendulum?

1.3 Discrete-time model:

The physical system evolves continuously in time:

$$\dot{x}(t) = f(x(t), u(t)).$$

However, a discrete-time (digital) controller does not update the control signal continuously. The measurements are obtained at discrete sampling instants

$$t_k = kT_s,$$

in which T_s is the sampling period, and a new control is computed every T_s seconds. Therefore, for designing a controller that operates in this manner we need a model that predicts the state from one sampling instant to another given by the discrete-time model

$$x[k+1] = A_d x[k] + B_d u[k],$$

in which $x[k] = x(kT_s)$. Therefore, the matrices depend on both the continuous-time model (in our case the linearized model) and the chosen sampling period.

To obtain a discrete model, it is necessary to specify what happens to the control input between two sampling instances. In this project, the selected method is called *zero-order hold (ZOH)* which means that the control input is kept constant during each interval, resulting in a piecewise-constant continuous time signal. Consider a one-dimensional system

$$\dot{x}(t) = ax(t) + bu(t),$$

assume that the input is held constant during one sampling interval as $u(t) = u[k]$ for $t \in [kT_s, (k+1)T_s)$, starting from $x[k]$, the state after one sampling period is

$$x[k+1] = e^{aT_s}x[k] + \left(\int_0^{T_s} e^{a\tau} b d\tau \right) u[k].$$

This corresponds to the discrete-time form

$$x[k+1] = a_d x[k] + b_d u[k],$$

with $a_d = e^{aT_s}$ and $b_d = \int_0^{T_s} e^{a\tau} b d\tau$.

For higher-dimensional state spaces, the same approach can be used by obtaining matrix exponential. You **do not** need to calculate these by hand, instead, you can use an appropriate MATLAB functionality.

In the Simulink diagram, the ZOH is implemented using the corresponding block. Inspect the diagram and identify the role of this block in the control system based on measurement and control samples.

2 Design a discrete-time LQR

For now, assume that both state variables $x_1[k]$ and $x_2[k]$ are measured. Design a discrete-time LQR that minimizes the cost

$$J = \sum_0^{\infty} (x[k]^T Q x[k] + u[k]^T R u[k]).$$

2.1 Task:

- Implement the resulting controller in the supplied MATLAB Function block in the Simulink model.
- The model takes in additional parameters the controller gain K , and actuation limits $umin$, and $umax$. These should be set in the accompanying .m file.

2.2 Experiments:

Evaluation: Evaluate the performance of your controller on the nonlinear model implemented in Simulink. Try different initial state values. Is it always possible to stabilize the system?

Choosing Q and R : Experiment with different different weight matrices Q and R . Investigate the effect on the settling time (how long does it take the pendulum to stabilize at the equilibrium), maximum angular displacement, control effort, and maximum control applied. How do you make a trade-off?

Actuator limits: LQR does not impose any constraints on the control. However, physical motors have limits on how much torque they can supply. Implement saturation of controls using the limits $umin$ and $umax$. How does the controller behavior change? Try different initial conditions, is it still possible to stabilize the system?

Disturbances: Add a small disturbance (noise) to the control input. The input to the physical system becomes

$$u[k] = -Kx[k] + v[k],$$

in which $v[k]$ is the realization of the disturbance at time k . You can use, **Add**, and **Random Number** blocks provided in the Simulink library for this purpose.

Model uncertainties: In the supplied MATLAB file, you will see two sets of parameters. Initially, the parameters used in the model (ones that start with **m.**, such as **m.m**, **m.l**) are identical to the ones used to simulate the actual system (ones that start with **p.**, such as **p.m**, **p.l**). Change these values so that there are discrepancies between the model and the actual system. Evaluate the performance of the controller.

3 Design an observer

So far, the full state is assumed to be measurable, that is, it was assumed that $y[k] = x[k]$. Now, consider a measurement model given by

$$y[k] = Cx[k],$$

with $C = \begin{bmatrix} 1 & 0 \end{bmatrix}$, meaning that the angular velocity is no longer measured. The controller you designed needs to rely on a state estimate, that is, $u[k] = -K\hat{x}[k]$.

3.1 Task:

- Design an observer and implement it in the provided MATLAB Function block in the Simulink model.
- Inspect the Simulink diagram. The observer block relies on the output of another block called **Unit Delay** which stores a discrete value for one sample, that is, it keeps track of the previous value. Reason about why this was needed in the simulation.

3.2 Experiments:

Observer poles: Can you estimate the state? How do you determine that? What considerations did you make when selecting appropriate values for the eigenvalues when designing the observer? Experiment with different values for the observer. Experiment when the initial estimate is incorrect.

Sampling time: Vary the sampling time (T_s). How does it affect the performance of the observer and the controller? For this task, you can also compare the performance of the controller and the observer separately using the full state measurements and running the observer in parallel.

Measurement disturbances: Consider applying a small noise to the measurements. Now, the measurement model is given by

$$y[k] = Cx[k] + \omega[k],$$

in which ω is a measurement noise. How does the controller behave under the measurement noise?

Alternative estimation methods: What other ways can you think of for estimating the angular velocity, given the angle? How do you think this would compare to the observer you designed?